

Data Replication in a Single-Leader Environment

MongoDB Replica Set + Python Control Node

Mert Karahan Doğukan Topçu

CENG465 — Principles of Data-Intensive Systems

May 5, 2026

Project Overview

Goal: demonstrate single-leader replication on a truly distributed setup — two physical macOS machines, no containers.

What we built

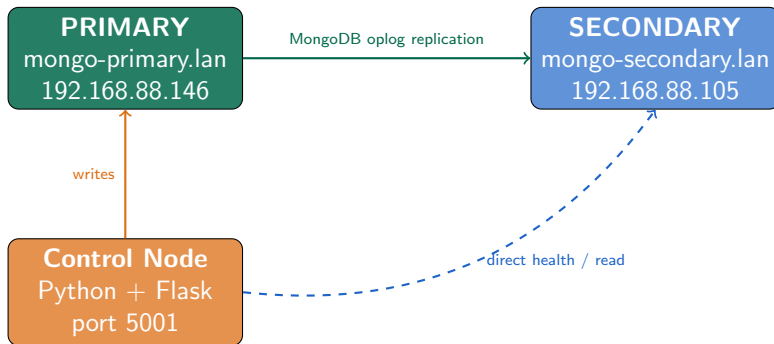
- MongoDB replica set on two machines (LAN)
- Replication-aware data schema
- Operation logging (insert / update / delete)
- Write concern toggle: `w=majority` vs `w=1`
- Dashboard: health, delay, and sync state

Central Question

What happens when the follower is unavailable?

- Should a write **block**?
- Should it be **queued** on the leader?
- How do we **prove** catch-up?

Deployment Architecture



Distributed Requirement

Database nodes run on separate physical macOS machines on the same LAN — not containers on one machine.

Replica Set Configuration

Initial issue

- Two voting members: primary + secondary
- Disconnecting secondary removed majority
- MongoDB reported `ReplicaSetNoPrimary`
- Even `w=1` could not write — no writable primary

Solution: non-voting read replica

- Primary: `votes=1, priority=2`
- Secondary: `votes=0, priority=0`

Primary stays writable during follower outages.

```
cfg = rs.conf()
cfg.members[1].priority = 0
cfg.members[1].votes = 0
rs.reconfig(cfg)
```

Schema Design

items collection

```
{
  key,
  value,
  version,           -- increments on each write
  leader_term,
  last_log_index,
  last_operation_id,
  last_updated,
  deleted            -- soft delete
}
```

operation_logs collection

```
{
  operation_id,
  operation_type,
  log_index,
  leader_write_time,
  follower_visible_time,
  replication_delay_ms,
  version_before,
  version_after,
  status,           -- pending / visible / timeout
  write_concern
}
```

Raft-inspired metadata

leader_term, log_index, operation_id, and version make update ordering and follower visibility fully observable.

Write Concern Modes

Mode	Secondary Up	Secondary Down
w=majority	Write succeeds; replication delay is measured and logged	Write rejected before primary mutation — no data change
w=1	Write succeeds immediately; log is reconciled in background	Write succeeds on primary; log stays pending_follower until catch-up

w=majority — Consistency-first

Reject the write rather than risk divergence.

w=1 — Availability-first

Accept the write; guarantee eventual delivery via oplog.

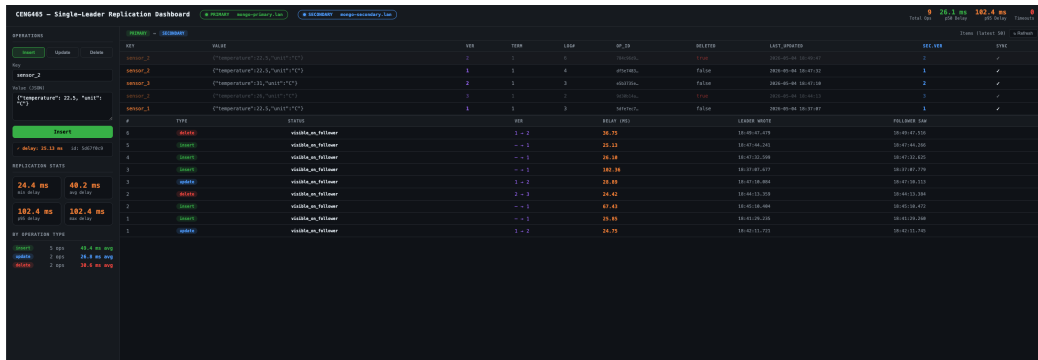
Async Recovery Flow ($w=1$)

- 1 User switches to $w=1$ on the dashboard.
- 2 Secondary network is disconnected.
- 3 Write goes to primary; log records `pending_follower`.
- 4 *Already-synced rows remain marked synced* (proven via prior logs).
- 5 Secondary reconnects.
- 6 MongoDB replays the oplog; secondary catches up.
- 7 Control node reconciler detects version / log-index match.
- 8 Log becomes `visible_on_follower`; sync checkmark appears.

Key design decisions

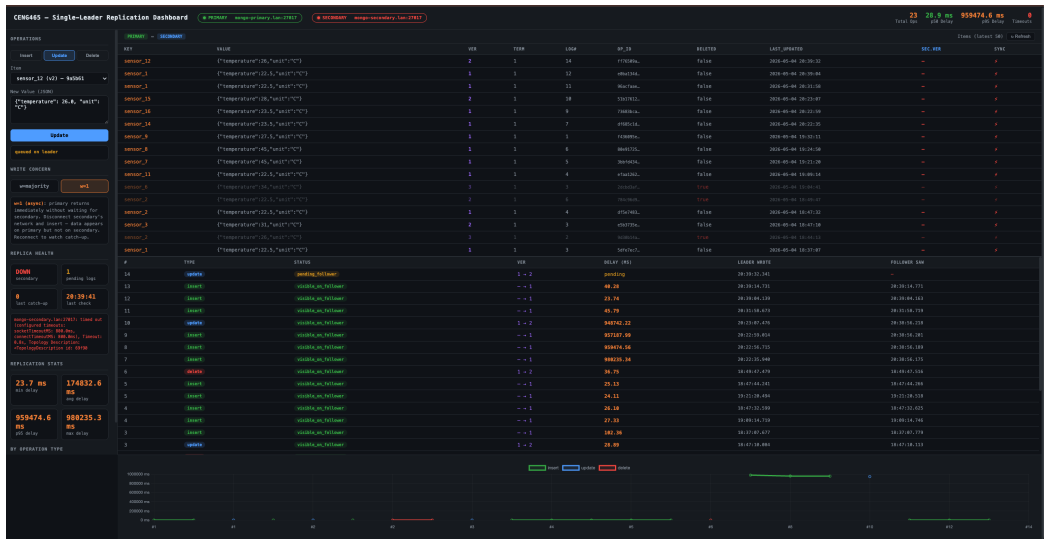
- Secondary health uses `directConnection=true`
- Reconciler runs every second in background thread
- Fallbacks: `version ≥ expected` **or** `log-index ≥ expected`

Dashboard



Left panel: operations + write concern toggle. Right: item sync state (primary ver vs. secondary ver). Bottom: replication delay chart.

Async Replication Evidence



w=1 active. New item written while secondary offline: primary version 1, secondary version --- (not yet)

All manual tests passed

- `w=majority` + secondary up: write visible on follower
- `w=majority` + secondary down: write rejected, no primary mutation
- `w=1` + secondary down: primary write succeeds (`pending_follower`)
- Reconnect: pending logs reconciled, sync restored
- Old synced rows stay synced during outage

What we proved

- Replication is **asynchronous by default**
- Write concern controls the **durability–availability tradeoff**
- Catch-up is driven by the **MongoDB oplog**, not the application
- **Observability** (version, log-index, delay) is essential for debugging distributed state